

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	Poornvaja B	Reg.No.:	192473008
Department:	BECSE - (DS)	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I B. Poorwaja (192473008) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

B. Poorwaja

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	4	4	5	4	2	19
Q2	4	3	4	3	2	16
Q3	3	2	3	4	2	14
Q4	4	4	4	3	2	17
Q5	3	3	5	4	2	17

83

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0)

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	B.Poorvaja	Reg.No.:	192473008
Department:	B.E-CSE(DS)	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Signed Integer Addition Using 2's Complement

Problem Analysis

The program performs signed integer addition using 8-bit 2's complement representation. Consider:

- First number = +120
- Second number = +50
- 8-bit signed range = -128 to +127

Binary Representation

+120:

120 = 01111000

+50:

50 = 00110010

Adding them:

01111000 (+120)

+ 00110010 (+50)

10101010

RESULT: 10101010

Since the MSB is 1, this is interpreted as a negative number in 2's complement.

Magnitude: 10101010

Invert $\rightarrow$ 01010101

Add 1 $\rightarrow$ 01010110 = 86

Therefore: **10101010 = -86**

But mathematically:

$120 + 50 = 170$

The correct result 170 cannot be represented in 8-bit signed 2's complement, because the maximum value is +127.

ERROR IDENTIFIED

The error is signed integer overflow.

Overflow occurs when two positive numbers are added and the result becomes negative.

DEBUGGED SOLUTION

The program should check for overflow after addition.

For signed integers:

If $A > 0$ and $B > 0$ and $\text{Result} < 0 \rightarrow$ Positive overflow

If $A < 0$ and $B < 0$ and $\text{Result} \geq 0 \rightarrow$ Negative overflow

Another hardware-based method is to compare the carry into the sign bit with the carry out of the sign bit:

$\text{Overflow} = \text{Carry into MSB} \text{ XOR } \text{Carry out of MSB}$

OPTIMIZATION

Use a wider integer representation when larger values are expected.

For example:

$120 + 50 = 170$

can be represented correctly using 16-bit signed arithmetic.

CONCLUSION

The incorrect result is not caused by an error in binary addition. It is caused by 8-bit signed overflow. The program should detect overflow and either report an error or use a wider data type.

2.A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

RIPPLE CARRY ADDER VS CARRY LOOK-AHEAD ADDER

PROBLEM ANALYSIS

A ripple carry adder performs addition one bit at a time. The carry generated at one bit must propagate to the next bit.

For example, in a 4-bit addition:

A3 A2 A1 A0
B3 B2 B1 B0
↓
FA0
↓ Carry
FA1
↓ Carry
FA2
↓ Carry
FA3

Each full adder must wait for the carry from the previous stage.

SOURCE OF DELAY

A = 1111

B = 0001

The carry propagates through several stages:

1111

0001

10000

The carry travels from:

Bit 0 → Bit 1 → Bit 2 → Bit 3 → Carry out

Therefore, the total delay increases as the number of bits increases.

DEBUGGING THE BOTTLENECK

The major bottleneck is sequential carry propagation.

For an n-bit ripple carry adder: Delay $\propto n$

Hence, a 32-bit or 64-bit ripple carry adder can introduce significant delay.

OPTIMIZED SOLUTION: CARRY LOOK-AHEAD ADDER

A carry look-ahead adder calculates carries in parallel using **Generate (G)** and **Propagate (P)** signals.

For each bit:

$$P_i = A_i \text{ XOR } B_i$$

$$G_i = A_i \text{ AND } B_i$$

$$\text{The carry is: } C_{i+1} = G_i + P_i C_i$$

For a 4-bit CLA:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Thus, carries can be calculated without waiting for each previous full adder.

COMPARISON

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry calculation	Sequential	Parallel
Speed	Slow	Fast
Hardware	Simple	More complex
Delay	High	Low
Suitable for	Simple circuits	High-speed processors

CONCLUSION

The ripple carry adder's performance problem is caused by carry propagation delay. Replacing it with a carry look-ahead adder allows carries to be generated in parallel, significantly reducing arithmetic execution time.

3.A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

BOOTH'S ALGORITHM FOR SIGNED MULTIPLICATION

PROBLEM ANALYSIS

Booth's algorithm is used for multiplying signed binary numbers using 2's complement representation. The assessment specifically highlights errors involving bit-pair checking, arithmetic shifts, and sign extension.

The algorithm examines: (Q0, Q-1)

BOOTH'S DECISION TABLE

Q0 Q-1	Operation
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

Where:

- A = accumulator
- M = multiplicand
- Q = multiplier
- Q-1 = extra bit

POSSIBLE DEBUGGING ERROR 1: INCORRECT BIT PAIR

The program must check: (Q0, Q-1) and not:(Q1, Q0)

For example: Q0 Q-1 = 01 means $A = A + M$

while: Q0 Q-1 = 10 means $A = A - M$

Reversing these bits produces an incorrect result.

POSSIBLE DEBUGGING ERROR 2: LOGICAL INSTEAD OF ARITHMETIC SHIFT

Booth's algorithm uses an arithmetic right shift.

For example:

Before: **10110010**

Arithmetic right shift: **11011001**

The sign bit 1 is copied into the new MSB.

A logical shift would produce: **01011001**

which changes the sign and gives an incorrect result.

POSSIBLE DEBUGGING ERROR 3: INCORRECT SIGN EXTENSION

When a negative value is shifted or extended, the sign bit must be preserved.

For example: **8-bit: 11110110**

Sign-extended to 16 bits: **11111111 11110110**

It must not become: **00000000 11110110**

CORRECT DEBUGGING PROCEDURE

1. Initialize $A = 0$.
2. Load the multiplicand into M.
3. Load the multiplier into Q.
4. Set $Q-1 = 0$.
5. Examine $(Q_0, Q-1)$.
6. Perform addition/subtraction when required.
7. Perform an arithmetic right shift on $(A, Q, Q-1)$.
8. Repeat for the required number of bits.
9. Combine A and Q to obtain the final product.

OPTIMIZATION

Use a fixed-width register system and perform explicit sign extension before arithmetic operations. This prevents incorrect results when multiplying negative operands.

CONCLUSION

The most likely causes of incorrect Booth multiplication are:

- Reversing the $Q_0, Q-1$ pair.
- Performing logical instead of arithmetic shifts.
- Incorrect 2's complement subtraction.
- Failing to sign-extend negative operands.
- Using an incorrect number of iterations.

Correcting these operations ensures accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

RESTORING DIVISION VS NON-RESTORING DIVISION

PROBLEM ANALYSIS

The restoring division algorithm repeatedly performs subtraction. If the subtraction produces a negative result, the previous value is restored by adding the divisor again. This repeated restoration increases execution time.

RESTORING DIVISION WORKFLOW

The basic process is:

SHIFT



SUBTRACT DIVISOR



IS REMAINDER NEGATIVE?



YES → RESTORE PREVIOUS REMAINDER



SET QUOTIENT BIT

The problem is the additional restoration operation.

Example of Unnecessary Operation

After subtraction: $A = A - M$

produces: $A < 0$

The restoring algorithm performs: $A = A + M$

to restore the previous value.

Then it continues with the next iteration.

This additional addition consumes processing time.

SOURCE OF INEFFICIENCY

The main problem is:

SUBTRACTION



NEGATIVE?



RESTORE USING ADDITION

Thus, some iterations require both subtraction and restoration addition.

OPTIMIZED SOLUTION: NON-RESTORING DIVISION

Non-restoring division avoids immediately restoring the negative remainder.

Instead: If $A \geq 0$: Shift and subtract M

If $A < 0$: Shift and add M

The sign of the current partial remainder determines the operation in the next iteration.

COMPARISON

Feature	Restoring Division	Non-Restoring Division
Negative remainder	Immediately restored	Not immediately restored
Extra addition	Frequently required	Avoided
Speed	Slower	Faster
Hardware operations	More	Fewer
Efficiency	Lower	Higher

ADVANTAGES OF NON-RESTORING DIVISION

- Eliminates unnecessary restoration operations.
- Reduces the number of arithmetic operations.
- Improves execution speed.
- Is suitable for high-performance arithmetic units.
- Maintains correct quotient and remainder computation.

CONCLUSION

The bottleneck in restoring division is the repeated restore operation after a negative partial remainder. Non-restoring division improves performance by allowing negative partial remainders to continue to the next iteration and deciding the next operation based on their sign.

5.A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

IEEE 754 SINGLE-PRECISION FLOATING-POINT ADDITION

PROBLEM ANALYSIS

IEEE 754 single precision uses 32 bits:

Sign	Exponent	Fraction
1	8	23

The problem occurs when a very small number is added to a very large number. The assessment specifically requires analysis of exponent alignment, normalization and rounding.

STEP 1: COMPARE EXPONENTS

Before addition, the exponents of both numbers must be made equal.

Example:

Large number: 1.101×2^{10}

Small number: 1.011×2^2

The smaller exponent must be increased:

$$1.011 \times 2^2$$

↓

$$0.000001011 \times 2^{10}$$

STEP 2: ADD THE MANTISSAS

1.101000000
+ 0.000001011

1.101001011

The result is then normalized.

STEP 3: NORMALIZATION

The result must follow the form: $1.xxxxx \times 2^n$

If the result is: 10.110×2^5

it must be normalized: 1.0110×2^6

The exponent is increased accordingly.

STEP 4: ROUNDING

Single precision has only 23 fraction bits. Therefore, if additional bits are produced, the result has to be rounded. This can cause small numerical differences.

WHY VERY SMALL NUMBERS CAN DISAPPEAR

Consider: Large number = 1.000000×2^{20}

Small number = 1.000000×2^{-20}

The smaller number must be shifted by a very large number of positions to align its exponent.

Eventually, its significant bits may fall outside the 23-bit precision available.

Large + Small $\approx$ Large

The small value may have little or no effect on the stored result.

SOURCES OF ERROR

The possible causes are:

1. **Exponent alignment**
 - Large exponent differences cause significant shifting.
2. **Limited mantissa precision**
 - Only 23 fraction bits are available.
3. **Rounding**
 - Extra bits must be discarded according to the rounding mode.
4. **Normalization**
 - Incorrect normalization can produce an incorrect exponent or mantissa.
5. **Loss of significance**
 - Small values may disappear when added to much larger values.

DEBUGGED SOLUTION

The floating-point addition implementation should:

1. Compare exponents
2. Align the smaller exponent
3. Add/subtract significands
4. Normalize the result
5. Round correctly
6. Check overflow/underflow
7. Store the final IEEE 754 representation

OPTIMIZATION TECHNIQUES

1. Use Double Precision

Double precision provides: 64 bits with a much larger significand than single precision. Therefore, it can represent small differences more accurately.

2. Use Numerically Stable Algorithms

For repeated calculations, reorganizing the computation can reduce accumulated rounding errors.

3. Avoid Repeated Lossy Operations

Instead of repeatedly rounding intermediate values, retain higher precision until the final stage.

4. Use Appropriate Summation Methods

For a large number of floating-point values, techniques such as compensated summation can reduce accumulated rounding error.

COMPARISON

Feature	Single Precision	Double Precision
Total bits	32	64
Exponent bits	8	11
Fraction bits	23	52
Accuracy	Lower	Higher
Memory usage	Lower	Higher
Suitable for	General calculations	Scientific/high-precision calculations

CONCLUSION

The inaccurate result is mainly caused by limited precision, exponent alignment, normalization and rounding. Using double precision and numerically stable algorithms can significantly reduce the loss of accuracy.

Code of Conduct:

I _B.Poorvaja(192473008)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0)